Digital image processing project 2

——watermarking

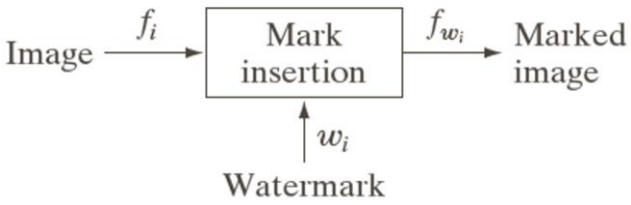
Goals

The goals of this project are as follows:

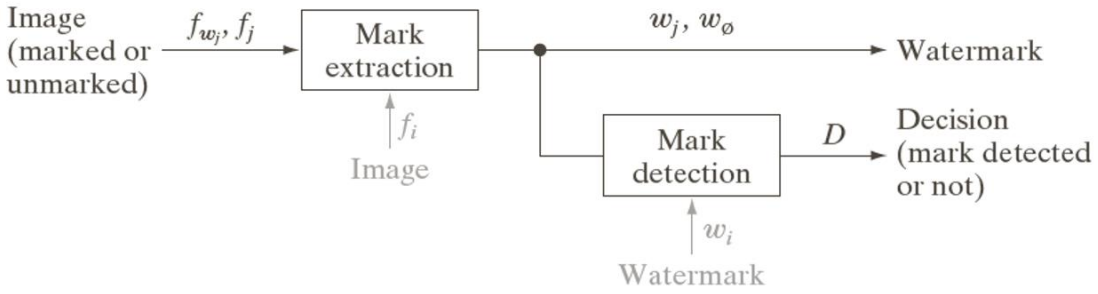
1. Understand digital image watermarking.
2. Get practice with creating fragile or robust invisible watermark.
3. Learn to design encoder and decoder of your own image watermarking system.
4. Try to attack other's watermarking system and withstand attacks from others.
5. Design solution for complex problem that meet a combination of societal, user, business and customer need as appropriate.

Description

In this project, both image and watermark should be chosen randomly from the given image set. You should design encoder and decoder of your own watermarking system (Fig.1), and submit THREE functions EXACTLY as defined in Function Declaration below. These three functions should be in three separate Matlab m files, respectively. Make sure these functions work well in different versions of Matlab (If not, please specify which version is required to run these functions).



(a)



(b)

Figure 1 A typical image watermarking system (a) encoder (b) decoder

Function Declaration

1. Function One: Encoding

function [*WatermarkedImg Scale*] = Encoding_GroupID(*Image*, *Watermark*)

% Inserting *Watermark* image into *Image*. The *Watermark* image should be invisible in the output *WatermarkedImg* image. The perceptual difference between *Image* and *WatermarkedImg* should be as small as possible.

% *Image* Input, RGB color image of size 512*512*3, uint8.

% *Watermark* Input, RGB color image of size 140*200*3, uint8.

% *WatermarkedImg* Output, RGB color image of size 512*512*3, uint8.

% *Scale* Output, a scalar, double, reserved for some encoding algorithm. Set it to 0, if you don't use it.

'GroupID' should be replaced with your actual group ID (e.g. Group 5 is supposed to submit a m file named Encoding_5.m, and the encoding function should be defined as 'function *WatermarkedImg* = Encoding_5(*Image*, *Watermark*)').

2. Function Two: Decoding

function [*DetectedWmk Flag*] = Decoding_GroupID(*Image*, *OriImg*, *OriWmk*, *Scale*)

% Recovering *Watermark* image from *Image* if the original watermark *OriWmk* is detected in *Image*, otherwise output an empty matrix.

% *Image* Input, RGB color image of size 512*512*3, uint8. It may be marked or unmarked image.

% *OriImg* Input, RGB color image of size 512*512*3, uint8. Original unmarked image. The content of and *Image* are *OriImg* the same, which means and they could be exactly the same image, or they just look the same (*Image* could be the output of your own encoding function described before or it could be a modified version of the original image but contain no watermark information). If your algorithm doesn't need the original image in the decoding stage, just leave it where it is.

% *OriWmk* Input, RGB color image of size 140*200*3, uint8. Original watermark. If your algorithm doesn't need the original watermark in the decoding stage, just leave it where it is.

% *Scale* Input, a scalar, double, reserved for some decoding

algorithm. Just leave it where it is, if you don't use it.

% *DetectedWmk* Output, RGB color image of size 140*200*3, uint8, if *Flag* = 1; empty matrix (*DetectedWmk* = []) if *Flag* = 0.

% *Flag* Output, logical or double, the value of *Flag* should be 1 if watermark is detected, and it should be 0 otherwise.

'GroupID' should be replaced with your actual group ID (e.g. Group 5 is supposed to submit a m file named Encoding_5.m, and the encoding function should be defined as 'function [*DetectedWmk Flag*]= Decoding_5(*Image*, *OriImg*, *OriWmk*)').

3. Function Three:Attacking

function *AttackedImg* = Attacking_GroupID(*Image*)

% Destroy the potential watermark contained in *Image* without changing the appearance of input *Image*. Geometric modification is NOT allowed, including but not limited to cropping, reflection, scaling, shearing, rotation, affine transformation and perspective transformation. The perceptual difference between *Image* and *AttackedImg* should be as small as possible.

% *Image* Input, RGB color image of size 512*512*3, uint8.

% *AttackedImg* Output, RGB color image of size 512*512*3, uint8.

'GroupID' should be replaced with your actual group ID (e.g. Group 5 is supposed to submit a m file named Attacking_5.m, and the attacking function should be defined as 'function *AttackedImg* = Attacking_5(*Image*)').

Discussion

Nowadays, more and more people live on live streams or short videos. Suppose a shorts-focused creator asks you to protect his videos from unauthorized modification and republishing, what are you going to do? Could your watermarking algorithm be used directly? What else should be done by our government or our law? What is the role of our DIP technique in it?

Submitting your work

1. Make a PPT files, and present your work in the class (Nov 5). Each group has 5~10 minutes.

2. Submit the three .m files to TA before 11:59, Nov 4

Make sure you can run main2026.m successfully with the three .m files you are going to submit (Check the bmp files generated by main2026.m).

3. Submit the following materials to TA before 23:59, Nov 13

A. Final Source code (three .m files)

B. Project report (in word or pdf format)

Evaluation

On the day of simulated confrontation, three steps will be followed to evaluate your work:

Step 1. Function One will be run to generate the watermarked image at first.

Step 2. The watermarked image generated in step 1 will then be attacked by Function Three provided by another group.

Step 3. The resulted image in step 2 will be passed to your Function Two along with the original image and watermark to generated the recovered watermark.

In step 1, if the watermark is visible in the output image, you will lose the chance to go to step 2 or 3. In step 2, your Function Three will be used to attack the watermarked images of other groups, and you will get a score for attacking (Score_Att). In step 3, the recovered watermark images will be shown in the QQ chat group anonymously and will be rated by all students. You will get a score for withstanding attack from others (Score_Def). Your final score is the average of Score_Att and Score_Def. In Addition, the best 2 attackers and the best 2 defenders will get a promotion for the second project.

Examples:

Suppose Group A is randomly chosen to withstand attack from Group B. Figure 2 shows the output image (in the middle) after running Function One of Group A, it also shows the output image (on the right) after running Function Three of Group B. Since the perceptual difference between the middle image and the right image is larger than the perceptual difference between the middle image and the left image, the Att value of Group B should be set to 0. If the perceptual difference between the middle and the right is smaller than the difference between the middle and the left, the Att value of Group B should be set to 2 (As shown in Fig. 3). Otherwise, it should be set to 1.

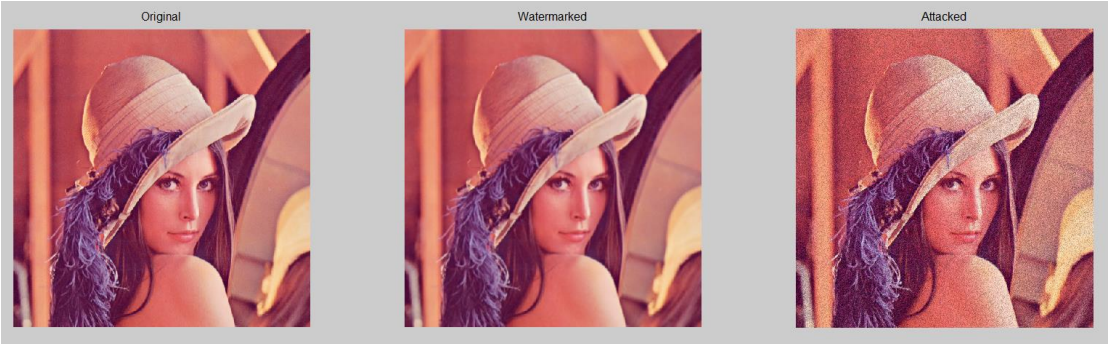


Figure 2. the original image(left), the watermarked image(middle), the attacked image(right)



Figure 3. the original image(left), the watermarked image(middle), the attacked image(right)

After running Function Two of Group A using the attacked image as input, the watermark is expected to be recovered if the attacked image contains watermark information. According to the quality of recovered watermark, the Def value of Group A will be set to one of {0,1,2} (Def = 2 if the recovered watermark can be easily distinguished from other watermarks in the given watermark file). With the Att value of Group B and Def value of Group A, the attacking score of Group B and defending score of Group A can be calculated by the following equation:

$$\text{Score_Att} = \text{Att} \times \max(\text{Att} - \text{Def}, 0),$$

$$\text{Score_Def} = \text{Def} \times \max(\text{Def} - \text{Att}, 0)$$

Att and Def of all groups are determined by all students.